

Specialized Neural Networks

Shortcomings in Artificial Neural Networks

- Some major shortcomings encountered in vanilla deep neural networks (DNNs) are:

Spatial Features Loss

- In DNNs, flattening a 2D image to a 1D vector input results in losing the spatial features of the image.
- Spatial features refer to the arrangement of the pixels in an image.
- This flattening leads to information loss because the network does not relate the pixel values to each other when trying to find patterns. The network has no knowledge of the fact that these pixel numbers were originally spatially arranged in a grid and that they are connected to each other.
- A specialized deep neural network called convolutional neural network (CNN) address this issue.
- CNNs do not require a flattened image, we can feed the raw image matrix of pixels to a CNN network, and the CNN will understand that pixels that are close to each other are more heavily related than pixels that are far apart.

Poor Scaling

- DNNs do not scale well for large images. refers to problems that occur when input data features have very different ranges or magnitudes.
- They are composed of dense layers that are fully connected to each other.
- While this is not a big problem for small images, but for large images the number of trainable parameters increases drastically with increase in image size.
- Also, full connectivity is wasteful, especially in images, and is prone to overfitting.
- Here also, CNNs come to the rescue. CNNs are locally connected layers, which use far fewer parameters than densely connected layers, as you will see

Failure to Handle Sequential Data

- They cannot capture sequential information in the input data, hence fail to handle sequence data.
- Another specialized deep neural network called recurrent neural network (RNN) addresses this issue.

Transfer Learning

- Training deep neural networks from scratch requires vast amounts of labeled data and significant computational resources, which is rarely practical.
- Collecting and manually labeling hundreds of thousands of images is costly, and training such models demands weeks of processing on multiple GPUs.
- Many researchers built state-of-the-art models that were trained on large standard image datasets and then shared their models with the general public for reuse.
- This means you should never have to train an image classifier from scratch again, unless you have an exceptionally large dataset and a very large computation budget to train everything from scratch by yourself.
- Even if that is the case, you might be better off using transfer learning to fine-tune the pre-trained network on your large dataset.
- Transfer learning is the transfer of the knowledge (feature maps) that the network has acquired from one task, where we have a large amount of data, to a new task where data is not abundantly available.
- The intuition behind transfer learning is that if a model is trained on a large and general enough dataset, this model will effectively serve as a generic representation of the visual world.

What is really being transferred from one network to another?

Answer: The network architecture and the trained weights.

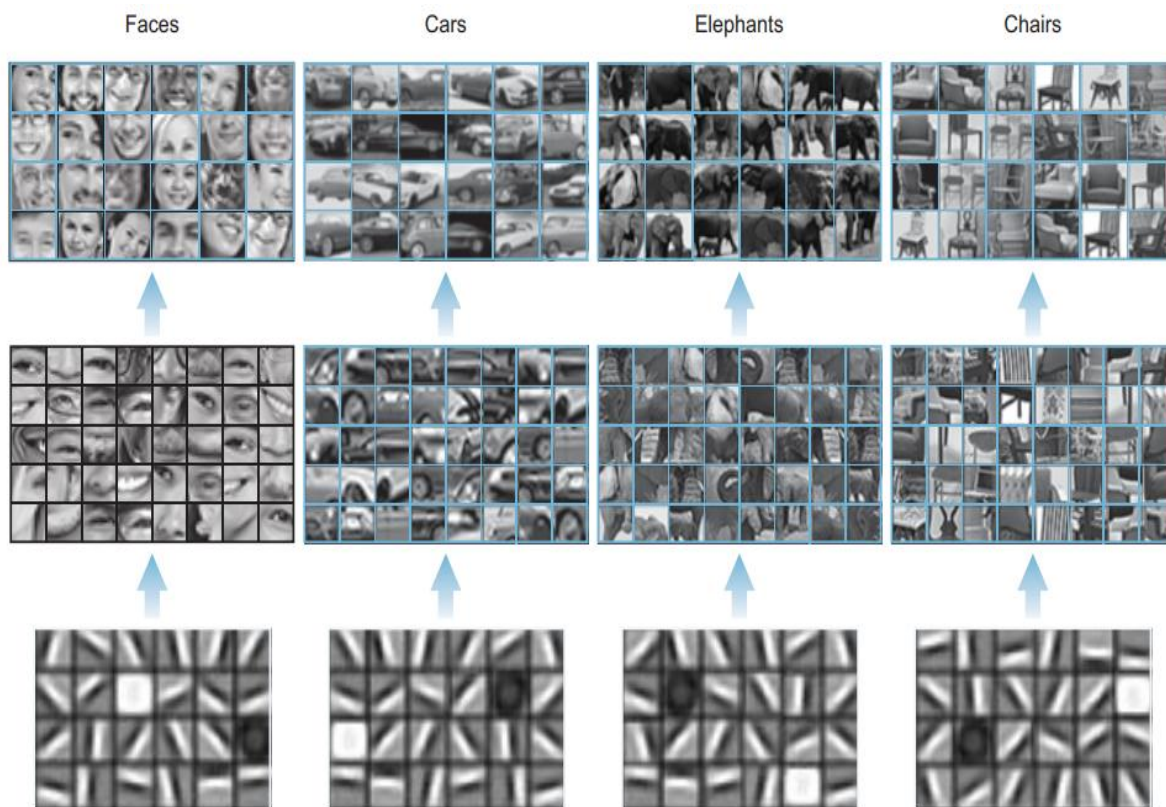
- These weights are then reused as the starting point for the training process and retrained to adapt to the new problem. During training, the model learns only the features that exist in our training dataset.
- These pre-trained models have spotted other features that might not be there in our dataset or our dataset might not be enough to learn all those things.
- By using a pre-trained network, we can basically download all this knowledge into our neural network to give it a huge and much faster start with even higher performance levels.

Why Transfer Learning Works

- In a neural network, features are learned hierarchically: starting from simple patterns to more complex structures.
- Early layers detect low-level features like edges and curves, while deeper layers combine these into higher-level patterns such as shapes, object parts, and eventually full objects.
- As the network goes deeper, it becomes more responsive to larger regions of the image and focuses on features most relevant for distinguishing between classes, while ignoring irrelevant details.
- The lower-level features are almost always transferable from one task to another because they contain generic information like the structure and nature of how images look.

Example : Detecting 4 classes – faces, cars, elephants and chairs

- Notice that the earlier layers' features are very similar for all the models.
- They represent low-level features like edges, lines, and blobs.
- This means models that are trained on one task capture similar relations in the data types in the earlier layers of the network and can easily be reused for different problems in other domains.
- The deeper the network, the more specific the features.

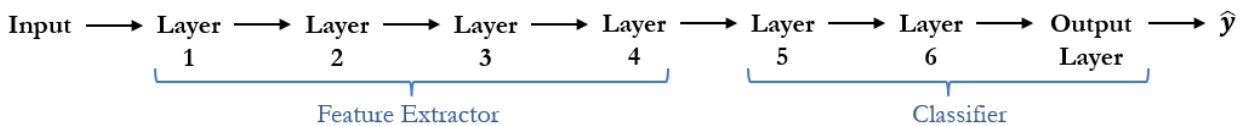


Transfer Learning Approaches

- A neural network model is first trained on a problem similar to the problem that is being solved.
- One or more layers from the trained model are then used in a new model trained on the problem of interest.
- This is motivated by the observation that the later layers of the network become progressively more specific to the details of the classes contained in the original dataset.
- Based on the similarity of the source and target domains, we can decide whether to transfer only the low-level features from the source domain, or the high-level features, or somewhere in between.
- The source domain is the original dataset that the pre-trained network is trained on.
- The target domain is the new dataset that we want to train the network on.
 1. There are three major transfer learning approaches: Pre-trained network as a classifier, pre-trained network as a feature extractor, and fine tuning.
- Each approach can be effective and save significant time in developing and training a deep CNN model.

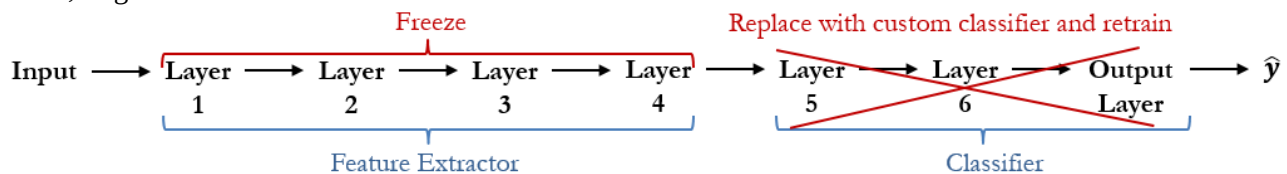
1. Using a Pre-trained Network as Classifier

- Use the whole network as it is with its trained weights.
- Download the network architecture and its pre-trained weights and then run the predictions directly on the new data.
- In this case, target domain is very similar to the source domain.



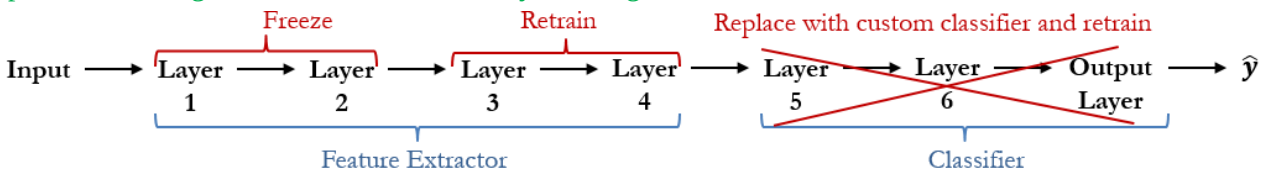
2. Using a Pre-trained Network as Feature Extractor

- Take a pre-trained CNN, freeze its feature extraction part, remove the classifier part, and add the customized new, dense classifier layers.
- Remove the classification part of the pre-trained network because it is often very specific to the original classification task, and subsequently it is specific to the set of classes on which the model was trained.
- The new layers will be trained from scratch, on top of the pre-trained model so that the previously learned feature maps can be reshaped for our dataset.
- In this case, target domain is somewhat similar to the source domain.



3. Fine Tuning

- Transfer learning remains effective even when the source and target domains differ significantly.
- By reusing and fine-tuning feature representations from a pre-trained model, we can adapt it to a new task.
- This involves freezing some early layers (which capture general features) and retraining the later layers along with a new classifier to fit the target domain.
- Fine-tuning is more efficient than training from scratch because it starts with already optimized weights rather than random ones.
- Random initialization offers no guarantee of starting near the optimal solution, often resulting in slower convergence.
- In contrast, fine-tuning adjusts pre-trained weights that are already close to useful representations, leading to faster training and better performance with less data and computation.
- Example 1: Retraining a few feature extraction layers along with the new classifier.



- Example 2: Retraining the whole network along with the new classifier.

